Building a Declarative UI

Part 2

Richard Powell, rmpowell77@me.com, v2.0

Previously



wxWidgets is a C++ library that lets developers create applications for Windows, macOS, Linux and other platforms with a single code base. It has popular language bindings for Python, Perl, Ruby and many other languages, and unlike other cross-platform toolkits, wxWidgets gives applications a truly native look and feel because it uses the platform's native API rather than emulating the GUI. It's also extensive, free, open-source and mature.

Previously

```
// Create the controls.
wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");

// Layout the controls.
wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

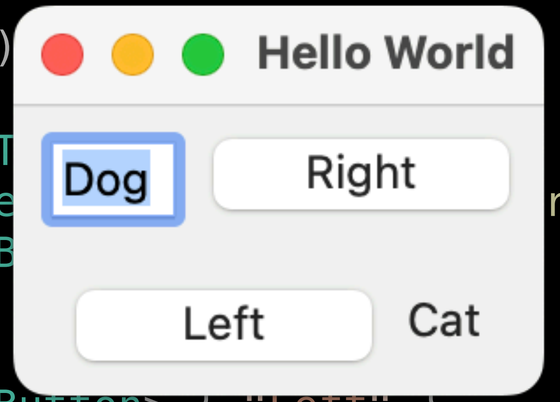
sizer->Add(sizerText, wxSizerFlags().Expand().Border());

wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

SetSizerAndFit(sizer);
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags()
    HSizer {
        Widget<wxT
            wxSize                                  rder() },
        Widget<wxB
    },
    HSizer {
        Widget<wxButton> { "Left" },
        Widget<wxStaticText> { "Cat" },
    },
}
    .attachTo(this);
```



Previously

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        Widget<wxTextCtrl> { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Widget<wxButton> { "Right" },
    },
    HSizer {
        Widget<wxButton> { "Left" },
        Widget<wxStaticText> { "Cat" },
    },
}
    .attachTo(this);
```

```
namespace DeclarativeUI {

template <typename T>
concept CreateAndAddable = requires(T widget, wxWindow* window, wxSize
    widget.createAndAdd(window, sizer, wxSizerFlags {});
};

template <typename W>
struct Widget {
    Widget(std::string str, std::optional<wxSizerFlags> flags = {})
        : str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags pa
    {
        sizer->Add(new W(parent, wxID_ANY, str), flags ? *flags : pare
    }

    std::string str;
    std::optional<wxSizerFlags> flags;
};

template <CreateAndAddable... W>
struct Sizer {
    Sizer(wxOrientation orientation, wxSizerFlags flags, W... widgets)
        : orientation(orientation)
        , flags(flags)
        , widgets(std::make_tuple(widgets...))
    {
    }

    Sizer(wxOrientation orientation, W... widgets)
        : orientation(orientation)
        , widgets(std::make_tuple(widgets...))
    {
    }
```

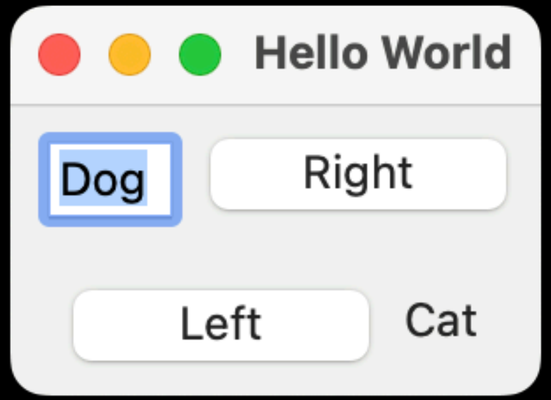
Previously

- Observer in your program sections that are *Imperative* and try to think *Declarative*.
  - Change the “How to do” parts into “What I want” code.
- Refactoring can cause Patterns to emerge.
- Know your types, algorithms, and emerging C++ constructs.

Shifting to be a Library Writer

- Understand the problem domain.
- Make your designs consistent, safe, and easy to use.
- Anticipate your users needs.

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        Widget<wxTextCtrl> { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Widget<wxButton> { "Right" },
    },
    HSizer {
        Widget<wxButton> { "Left" },
        Widget<wxStaticText> { "Cat" },
    },
}
    .attachTo(this);
```



```cpp
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        Widget<wxTextCtrl> { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Widget<wxButton> { "Right" },
    },
    HSizer {
        Widget<wxButton> { "Left" },
        Widget<wxStaticText> { "Cat" },
    },
}
    .attachTo(this);
```

```cpp
namespace DeclarativeUI {

template <typename W>
struct Widget {
    explicit Widget(std::string str, std::optional<wxSizerF
        : str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxS
    {
        sizer->Add(new W(parent, wxID_ANY, str), flags ? *f
    }

    std::string str;
    std::optional<wxSizerFlags> flags;
};

}
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        Widget<wxTextCtrl> { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Widget<wxButton> { "Right" },
    },
    HSizer {
        Widget<wxButton> { "Left" },
        Widget<wxStaticText> { "Cat" },
    },
}
    .attachTo(this);
```

```
namespace DeclarativeUI {

template <typename W>
struct Widget {
    explicit Widget(std::string str, std::optional<wxSizerF
        : str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxS
    {
        sizer->Add(new W(parent, wxID_ANY, str), flags ? *f
    }

    std::string str;
    std::optional<wxSizerFlags> flags;
};

using TextCtrl = Widget<wxTextCtrl>;
using Button = Widget<wxButton>;
using Text = Widget<wxStaticText>;

}
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
}
    .attachTo(this);
```

```
namespace DeclarativeUI {

template <typename W>
struct Widget {
    explicit Widget(std::string str, std::optional<wxSizerF
        : str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxS
    {
        sizer->Add(new W(parent, wxID_ANY, str), flags ? *f
    }

    std::string str;
    std::optional<wxSizerFlags> flags;
};

using TextCtrl = Widget<wxTextCtrl>;
using Button = Widget<wxButton>;
using Text = Widget<wxStaticText>;

}
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
}
    .attachTo(this);
```

```
namespace DeclarativeUI {

template <typename W>
struct Widget {
    explicit Widget(std::string str, std::optional<wxSizerF
        : str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxS
    {
        sizer->Add(new W(parent, wxID_ANY, str), flags ? *f
    }

    std::string str;
    std::optional<wxSizerFlags> flags;
};

using TextCtrl = Widget<wxTextCtrl>;
using Button = Widget<wxButton>;
using Text = Widget<wxStaticText>;

}
```

```cpp
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
}
    .attachTo(this);
```

```cpp
namespace DeclarativeUI::details {

template <typename W>
struct Widget {
    explicit Widget(std::string str, std::optional<wxSizerF
        : str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxS
    {
        sizer->Add(new W(parent, wxID_ANY, str), flags ? *f
    }

    std::string str;
    std::optional<wxSizerFlags> flags;
};

}

namespace DeclarativeUI {

using TextCtrl = details::Widget<wxTextCtrl>;
using Button = details::Widget<wxButton>;
using Text = details::Widget<wxStaticText>;

}
```

```cpp
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
}
    .attachTo(this);
```

```cpp
namespace DeclarativeUI::details {

template <typename W>
struct Widget {
    explicit Widget(std::string str, std::optional<wxSizerF
        : str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxS
    {
        sizer->Add(new W(parent, wxID_ANY, str), flags ? *f
    }

    std::string str;
    std::optional<wxSizerFlags> flags;
};

}

namespace DeclarativeUI {

using TextCtrl = details::Widget<wxTextCtrl>;
using Button = details::Widget<wxButton>;
using Text = details::Widget<wxStaticText>;

}
```

```cpp
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
}
    .attachTo(this);
```

```cpp
namespace DeclarativeUI::details {

template <typename W>
struct Widget {
    explicit Widget(std::string str, std::optional<wxSizerF
        : str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxS
    {
        sizer->Add(new W(parent, wxID_ANY, str), flags ? *f
    }

private:
    std::string str;
    std::optional<wxSizerFlags> flags;
};

}

namespace DeclarativeUI {

using TextCtrl = details::Widget<wxTextCtrl>;
using Button = details::Widget<wxButton>;
using Text = details::Widget<wxStaticText>;

}
```

```cpp
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

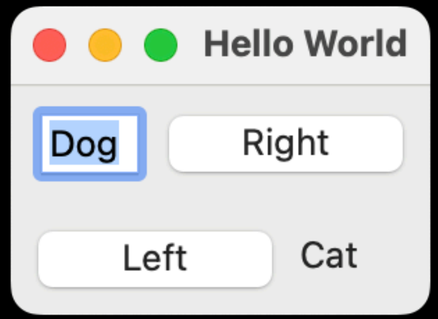
    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

    SetSizerAndFit(sizer);
}
```



```cpp
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    SetSizerAndFit(sizer);
}
```

```cpp
wxBEGIN_EVENT_TABLE(MyFrame, wxFrame)
    EVT_MENU(wxID_EXIT, MyFrame::OnExit)
    EVT_BUTTON(wxID_EXIT, MyFrame::OnExit)
wxEND_EVENT_TABLE()

void MyFrame::OnExit([[maybe_unused]] wxCommandEvent&
{
    Close(true);
}
```



```cpp
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    SetSizerAndFit(sizer);
}
```

```cpp
wxBEGIN_EVENT_TABLE(MyFrame, wxFrame)
    EVT_MENU(wxID_EXIT, MyFrame::OnExit)
    EVT_BUTTON(wxID_EXIT, MyFrame::OnExit)
wxEND_EVENT_TABLE()

void MyFrame::OnExit([[maybe_unused]] wxCommandEvent&
{
    Close(true);
}
```



```cpp
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
        wxLogMessage("Hello world from wxWidgets!");
    });

    SetSizerAndFit(sizer);
}
```

```cpp
wxBEGIN_EVENT_TABLE(MyFrame, wxFrame)
    EVT_MENU(wxID_EXIT, MyFrame::OnExit)
    EVT_BUTTON(wxID_EXIT, MyFrame::OnExit)
wxEND_EVENT_TABLE()

void MyFrame::OnExit([[maybe_unused]] wxCommandEvent&
{
    Close(true);
}
```



```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
}
    .attachTo(this);
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { "Exit" },
}
    .attachTo(this);
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { "Exit" },
}
    .attachTo(this);
```

```cpp
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { "Exit" },
}
    .attachTo(this);
```

```cpp
namespace DeclarativeUI::details {

template <typename W>
struct Widget {
    explicit Widget(std::string str, std::optional<wxSizerFl
        : str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSi
    {
        sizer->Add(new W(parent, wxID_ANY, str), flags ? *fl
    }

private:
    std::string str;
    std::optional<wxSizerFlags> flags;
};

}
```

```cpp
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { "Exit" },
}
    .attachTo(this);
```

```cpp
namespace DeclarativeUI::details {

template <typename W>
struct Widget {
    explicit Widget(wxWindowID id, std::string str, std::opt
        : id(id)
        , str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSi
    {
        sizer->Add(new W(parent, wxID_ANY, str), flags ? *fl
    }

private:
    wxWindowID id;
    std::string str;
    std::optional<wxSizerFlags> flags;
};

}
```

```cpp
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { "Exit" },
}
    .attachTo(this);
```

```cpp
namespace DeclarativeUI::details {

template <typename W>
struct Widget {
    explicit Widget(wxWindowID id, std::string str, std::opt
        : id(id)
        , str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSi
    {
        sizer->Add(new W(parent, wxID_ANY, str), flags ? *fl
    }

private:
    wxWindowID id;
    std::string str;
    std::optional<wxSizerFlags> flags;
};

}
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { "Exit" },
}
    .attachTo(this);
```

```
namespace DeclarativeUI::details {

template <typename W>
struct Widget {
    explicit Widget(wxWindowID id, std::string str, std::opt
        : id(id)
        , str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSi
    {
        sizer->Add(new W(parent, id, str), flags ? *flags :
    }

private:
    wxWindowID id;
    std::string str;
    std::optional<wxSizerFlags> flags;
};

}
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { "Exit" },
}
    .attachTo(this);
```

```
namespace DeclarativeUI::details {

template <typename W>
struct Widget {
    explicit Widget(wxWindowID id, std::string str, std::opt
        : id(id)
        , str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSi
    {
        sizer->Add(new W(parent, id, str), flags ? *flags :
    }

private:
    wxWindowID id;
    std::string str;
    std::optional<wxSizerFlags> flags;
};

}
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxID_ANY, "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { wxID_ANY, "Right" },
    },
    HSizer {
        Button { wxID_ANY, "Left" },
        Text { wxID_ANY, "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
namespace DeclarativeUI::details {

template <typename W>
struct Widget {
    explicit Widget(wxWindowID id, std::string str, std::opt
        : id(id)
        , str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSi
    {
        sizer->Add(new W(parent, id, str), flags ? *flags :
    }

private:
    wxWindowID id;
    std::string str;
    std::optional<wxSizerFlags> flags;
};

}
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxID_ANY, "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { wxID_ANY, "Right" },
    },
    HSizer {
        Button { wxID_ANY, "Left" },
        Text { wxID_ANY, "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
namespace DeclarativeUI::details {

template <typename W>
struct Widget {
    explicit Widget(wxWindowID id, std::string str, std::opt
        : id(id)
        , str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSi
    {
        sizer->Add(new W(parent, id, str), flags ? *flags :
    }

private:
    wxWindowID id;
    std::string str;
    std::optional<wxSizerFlags> flags;
};

}
```

```
HSizer {
    Button { wxID_ANY, "Left" },
    Text { wxID_ANY, "Cat" },
},
Button { wxID_EXIT, "Exit" },
```

```
explicit Widget(wxWindowID id = wxID_ANY, std::string str = std::string {}, std::optional<wxSizerFlags> flags = {})
    : id(id)
    , str(std::move(str))
    , flags(flags)
{
}

Widget(wxWindowID id, std::optional<wxSizerFlags> flags)
    : id(id)
    , flags(flags)
{
}

explicit Widget(std::string str, std::optional<wxSizerFlags> flags = {})
    : id(wxID_ANY)
    , str(std::move(str))
    , flags(flags)
{
}
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxID_ANY, "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { wxID_ANY, "Right" },
    },
    HSizer {
        Button { wxID_ANY, "Left" },
        Text { wxID_ANY, "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
explicit Widget(wxWindowID id = wxID_ANY, std::string str = s
    : id(id)
    , str(std::move(str))
    , flags(flags)
{
}

Widget(wxWindowID id, std::optional<wxSizerFlags> flags)
    : id(id)
    , flags(flags)
{
}

explicit Widget(std::string str, std::optional<wxSizerFlags>
    : id(wxID_ANY)
    , str(std::move(str))
    , flags(flags)
{
}
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
explicit Widget(wxWindowID id = wxID_ANY, std::string str = s
    : id(id)
    , str(std::move(str))
    , flags(flags)
{
}

Widget(wxWindowID id, std::optional<wxSizerFlags> flags)
    : id(id)
    , flags(flags)
{
}

explicit Widget(std::string str, std::optional<wxSizerFlags>
    : id(wxID_ANY)
    , str(std::move(str))
    , flags(flags)
{
}
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
explicit Widget(wxWindowID id = wxID_ANY, std::string str = s
    : id(id)
    , str(std::move(str))
    , flags(flags)
{
}

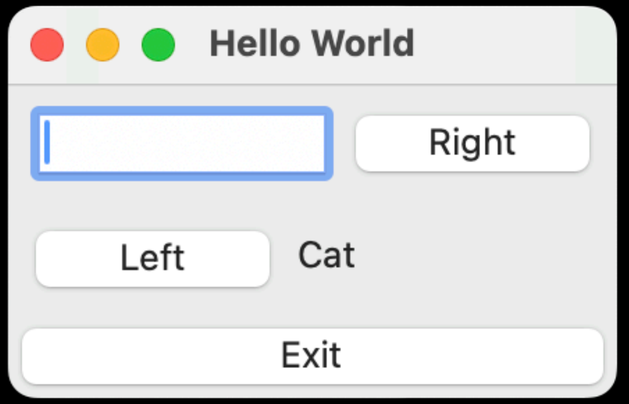
Widget(wxWindowID id, std::optional<wxSizerFlags> flags)
    : id(id)
    , flags(flags)
{
}

explicit Widget(std::string str, std::optional<wxSizerFlags>
    : id(wxID_ANY)
    , str(std::move(str))
    , flags(flags)
{
}
```

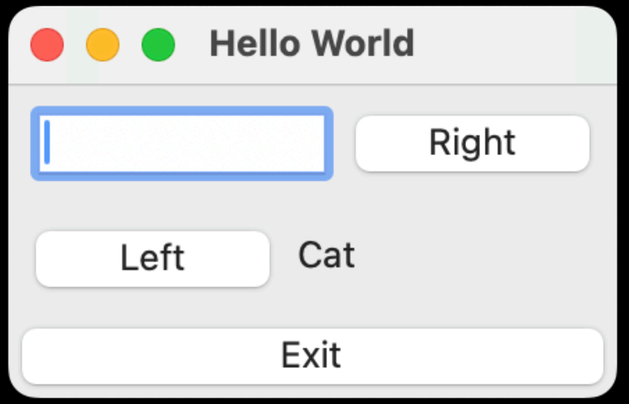
```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```



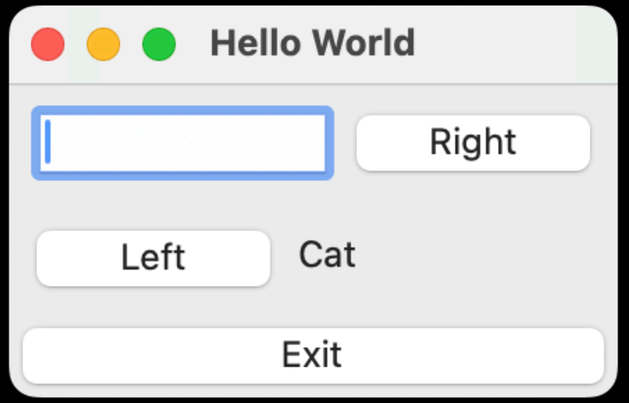
```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```



```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() },
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```








```cpp
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str), flags ? *flags : parentFlags);
    }

};
```

◆ wxTextCtrl() [2/2]

```
wxTextCtrl::wxTextCtrl ( wxWindow *           parent,
                         wxWindowID           id,
                         const wxString &     value = wxEmptyString ,
                         const wxPoint &      pos = wxDefaultPosition ,
                         const wxSize &       size = wxDefaultSize ,
                         long                 style = 0 ,
                         const wxValidator &  validator = wxDefaultValidator ,
                         const wxString &     name = wxTextCtrlNameStr
                       )
```

https://docs.wxwidgets.org/latest/classwx_text_ctrl.html

```cpp
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str), flags ? *flags : parentFlags);
    }

private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```

◆ wxTextCtrl() [2/2]

```
wxTextCtrl::wxTextCtrl ( wxWindow *           parent,
                         wxWindowID           id,
                         const wxString &     value = wxEmptyString ,
                         const wxPoint &      pos = wxDefaultPosition ,
                         const wxSize &       size = wxDefaultSize ,
                         long                 style = 0 ,
                         const wxValidator &  validator = wxDefaultValidator ,
                         const wxString &     name = wxTextCtrlNameStr
                       )
```

https://docs.wxwidgets.org/latest/classwx_text_ctrl.html

```cpp
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str), flags ? *flags : parentFlags);
    }

private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```

◆ wxTextCtrl() [2/2]

```
wxTextCtrl::wxTextCtrl ( wxWindow *           parent,
                         wxWindowID           id,
                         const wxString &     value = wxEmptyString ,
                         const wxPoint &      pos = wxDefaultPosition ,
                         const wxSize &       size = wxDefaultSize ,
                         long                 style = 0 ,
                         const wxValidator &  validator = wxDefaultValidator ,
                         const wxString &     name = wxTextCtrlNameStr
                       )
```

https://docs.wxwidgets.org/latest/classwx_text_ctrl.html

```cpp
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```

◆ wxTextCtrl() [2/2]

```cpp
wxTextCtrl::wxTextCtrl ( wxWindow *           parent,
                         wxWindowID           id,
                         const wxString &     value = wxEmptyString ,
                         const wxPoint &      pos = wxDefaultPosition ,
                         const wxSize &       size = wxDefaultSize ,
                         long                 style = 0 ,
                         const wxValidator &  validator = wxDefaultValidator ,
                         const wxString &     name = wxTextCtrlNameStr
                       )
```

https://docs.wxwidgets.org/latest/classwx_text_ctrl.html

```cpp
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```

◆ wxTextCtrl() [2/2]

```cpp
wxTextCtrl::wxTextCtrl ( wxWindow *           parent,
                         wxWindowID           id,
                         const wxString &     value = wxEmptyString ,
                         const wxPoint &      pos = wxDefaultPosition ,
                         const wxSize &       size = wxDefaultSize ,
                         long                 style = 0 ,
                         const wxValidator &  validator = wxDefaultValidator ,
                         const wxString &     name = wxTextCtrlNameStr
                       )
```

https://docs.wxwidgets.org/latest/classwx_text_ctrl.html

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    void withSize(wxSize size_)
    {
        size = size_;
    }


private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    void withSize(wxSize size_)
    {
        size = size_;
    }


private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
```

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    void withSize(wxSize size_)
    {
        size = size_;
    }


private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withSize(wxSize { 0, 0 }),
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
```

```
struct Widget {

    auto createAndAdd(wxWind
    {
        sizer->Add(new W(par
    }

    void withSize(wxSize siz
    {
        size = size_;
    }


private:
    wxPoint position = wxDef
    wxSize size = wxDefaultS
};



   using namespace Decla
   // Create and layout
   VSizer {
       wxSizerFlags().Ex
       HSizer {
           TextCtrl { wx
               .withSize
           Button { "Rig
       },
       HSizer {
           Button { "Lef
           Text { "Cat"
       },
```

```
[main] Building folder: /Users/richardpowell/Development/github.com/rmpowell77/
wxHelloWorld.git/build all
[build] Starting build
[proc] Executing command: /opt/homebrew/bin/cmake --build /Users/richardpowell/
Development/github.com/rmpowell77/wxHelloWorld.git/build --config Debug --target all --
[build] [1/2  50% :: 0.955] Building CXX object CMakeFiles/wxHelloWorld.dir/
HelloWorld.cpp.o
[build] FAILED: CMakeFiles/wxHelloWorld.dir/HelloWorld.cpp.o
[build] /usr/bin/clang++ -DWXUSINGDLL -D_FILE_OFFSET_BITS=64 -D__WXMAC__
-D__WXOSX_COCOA__ -D__WXOSX__ -isystem /opt/homebrew/lib/wx/include/osx_cocoa-
unicode-3.2 -isystem /opt/homebrew/include/wx-3.2 -g -std=c++20 -arch arm64 -isysroot /
Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/
MacOSX15.2.sdk -Wall -Wextra -Werror -MD -MT CMakeFiles/wxHelloWorld.dir/
HelloWorld.cpp.o -MF CMakeFiles/wxHelloWorld.dir/HelloWorld.cpp.o.d -o CMakeFiles/
wxHelloWorld.dir/HelloWorld.cpp.o -c /Users/richardpowell/Development/github.com/
rmpowell77/wxHelloWorld.git/HelloWorld.cpp
[build] /Users/richardpowell/Development/github.com/rmpowell77/wxHelloWorld.git/
HelloWorld.cpp:208:9: error: no viable constructor or deduction guide for deduction of
template arguments of 'HSizer'
[build]   208 |         HSizer {
[build]       |         ^
[build] /Users/richardpowell/Development/github.com/rmpowell77/wxHelloWorld.git/
HelloWorld.cpp:120:5: note: candidate function template not viable: cannot convert
argument of incomplete type 'void' to 'wxSizerFlags' for 1st argument
[build]   120 |     HSizer(wxSizerFlags flags, W... widgets)
[build]       |     ^      ~~~~~~~~~~~~~~~~~~
[build] /Users/richardpowell/Development/github.com/rmpowell77/wxHelloWorld.git/
HelloWorld.cpp:116:5: note: candidate template ignored: substitution failure [with W =
<void, Button>]: argument may not have 'void' type
[build]   116 |     HSizer(W... widgets)
[build]       |     ^          ~~~~~~~
[build] /Users/richardpowell/Development/github.com/rmpowell77/wxHelloWorld.git/
HelloWorld.cpp:114:8: note: candidate function template not viable: requires 1 argument,
but 2 were provided
[build]   114 | struct HSizer : details::Sizer<W...> {
[build]       |        ^~~~~
[build] 1 error generated.
[build] ninja: build stopped: subcommand failed.
[proc] The command: /opt/homebrew/bin/cmake --build /Users/richardpowell/Development/
github.com/rmpowell77/wxHelloWorld.git/build --config Debug --target all -- exited with
code: 1
[driver] Build completed: 00:00:00.991
[build] Build finished with exit code 1
```

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    void withSize(wxSize size_)
    {
        size = size_;
    }


private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withSize(wxSize { 0, 0 }),
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
```

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    auto withSize(wxSize size_) -> Widget<W>&
    {
        size = size_;
        return *this;
    }

private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withSize(wxSize { 0, 0 }),
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
```

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

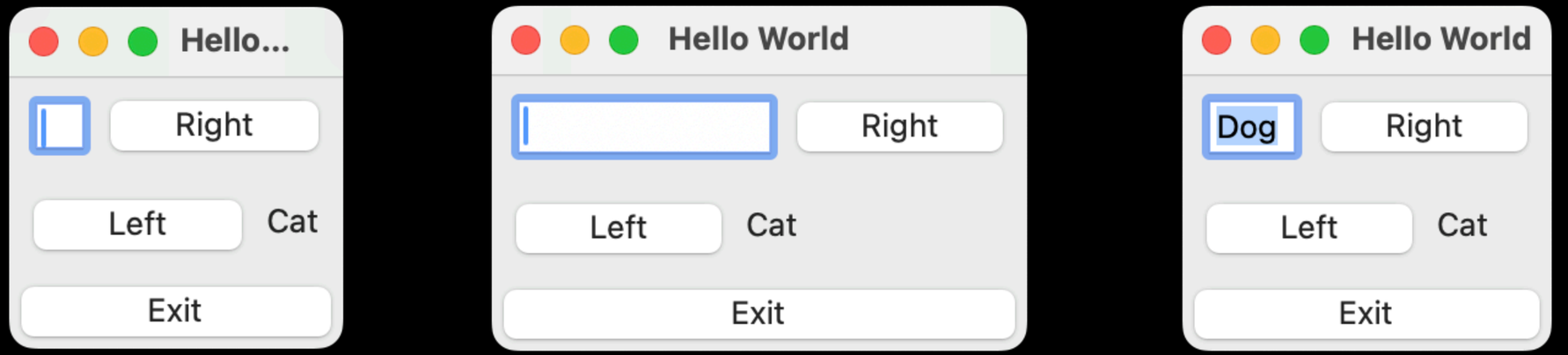
    auto withSize(wxSize size_) -> Widget<W>&
    {
        size = size_;
        return *this;
    }

private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```



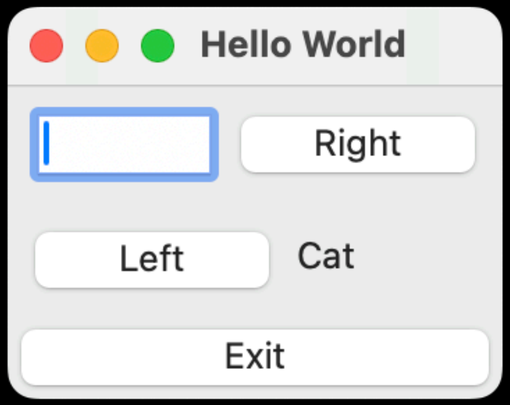

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withSize(wxSize { 0, 0 }),
        Button { "Right" },
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
```




Hello...
Right
Left
Cat
Exit
Hello World
Right
Left
Cat
Exit
Hello World
Dog
Right
Left
Cat
Exit




```
TextCtrl { wxSizerFlags(1).Expand().Border() }
    .withSize(wxSize { 0, 0 }),
```



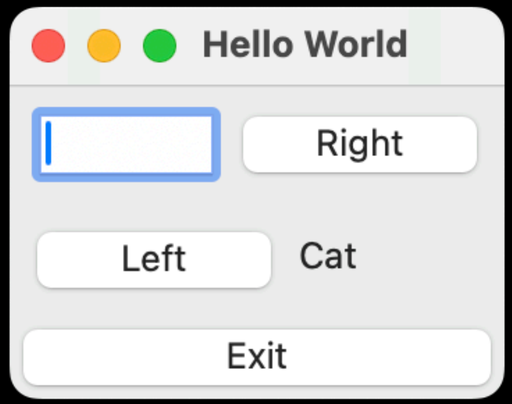

```
TextCtrl { wxSizerFlags(1).Expand().Border() }
    .withSize(wxSize { 64, -1 }),
```

```cpp
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    auto withSize(wxSize size_) -> Widget<W>&
    {
        size = size_;
        return *this;
    }

private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```




```cpp
TextCtrl { wxSizerFlags(1).Expand().Border() }
    .withSize(wxSize { 64, -1 }),
```

```cpp
template <typename W>
struct Widget {

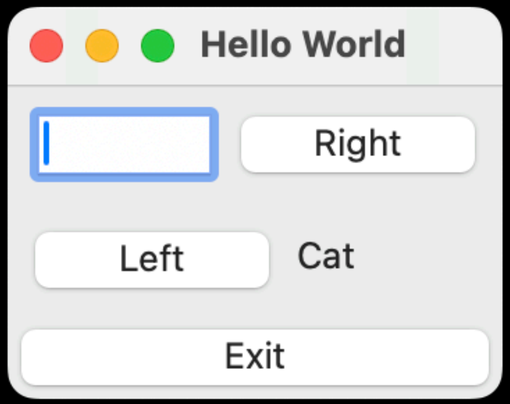
    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    auto withSize(wxSize size_) -> Widget<W>&
    {
        size = size_;
        return *this;
    }

    auto withWidth(int size_) -> Widget<W>&
    {
        size.SetWidth(size_);
        return *this;
    }

    auto withHeight(int size_) -> Widget<W>&
    {
        size.SetHeight(size_);
        return *this;
    }

private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```




```cpp
TextCtrl { wxSizerFlags(1).Expand().Border() }
    .withSize(wxSize { 64, -1 }),
```

```cpp
template <typename W>
struct Widget {

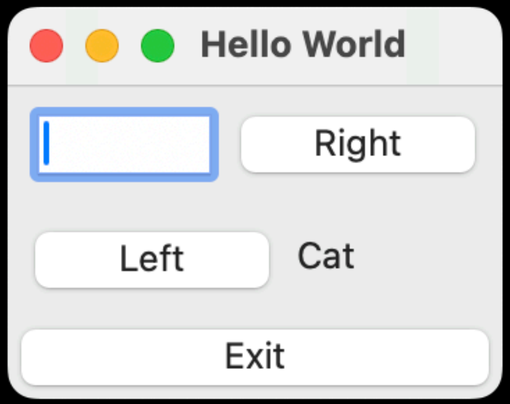
    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    auto withSize(wxSize size_) -> Widget<W>&
    {
        size = size_;
        return *this;
    }

    auto withWidth(int size_) -> Widget<W>&
    {
        size.SetWidth(size_);
        return *this;
    }

    auto withHeight(int size_) -> Widget<W>&
    {
        size.SetHeight(size_);
        return *this;
    }

private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```




```cpp
TextCtrl { wxSizerFlags(1).Expand().Border() }
    .withSize(wxSize { 64, -1 }),
```

```cpp
template <typename W>
struct Widget {

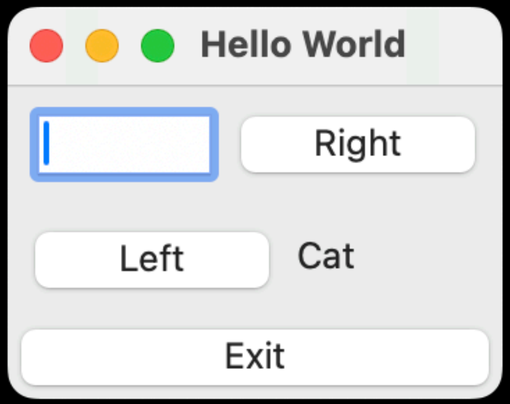
    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    auto withSize(wxSize size_) -> Widget<W>&
    {
        size = size_;
        return *this;
    }

    auto withWidth(int size_) -> Widget<W>&
    {
        size.SetWidth(size_);
        return *this;
    }

    auto withHeight(int size_) -> Widget<W>&
    {
        size.SetHeight(size_);
        return *this;
    }

private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```




```cpp
TextCtrl { wxSizerFlags(1).Expand().Border() }
    .withWidth(64),
```

```cpp
template <typename W>
struct Widget {

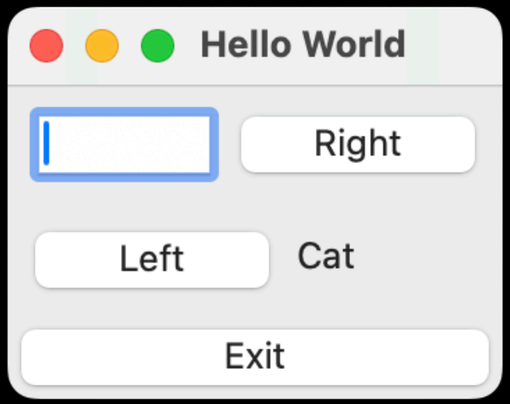
    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    auto withSize(wxSize size_) -> Widget<W>&
    {
        size = size_;
        return *this;
    }

    auto withWidth(int size_) -> Widget<W>&
    {
        size.SetWidth(size_);
        return *this;
    }

    auto withHeight(int size_) -> Widget<W>&
    {
        size.SetHeight(size_);
        return *this;
    }

private:
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
};
```




```cpp
TextCtrl { wxSizerFlags(1).Expand().Border() }
    .withWidth(64),
```

Method chaining

aka Builder Pattern

```
auto withSize(wxSize size_) -> Widget<W>&
{
    size = size_;
    return *this;
}
```

```
Button { "Exit" }
    .withColor(RED)
    .withWidth(16)
    .withHelp(),
```

```
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    SetSizerAndFit(sizer);
}
```

```
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
        wxLogMessage("Hello world from wxWidgets!");
    });

    SetSizerAndFit(sizer);
}
```

```
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
        wxLogMessage("Hello world from wxWidgets!");
    });

    SetSizerAndFit(sizer);
}
```

```
wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");

btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
    wxLogMessage("Hello world from wxWidgets!");
});
```

```
            wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");

            btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
                wxLogMessage("Hello world from wxWidgets!");
            });
```

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

};
```

```
            wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");

            btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
                wxLogMessage("Hello world from wxWidgets!");
            });
```

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    using Handler = std::function<void(wxCommandEvent&)>;
    auto bind(Handler handler) -> Widget<W>&
    {
        boundedHandler = handler;
        return *this;
    }

private:
    std::optional<Handler> boundedHandler;
};
```

```
            wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");

            btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
                wxLogMessage("Hello world from wxWidgets!");
            });
```

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
    }

    using Handler = std::function<void(wxCommandEvent&)>;
    auto bind(Handler handler) -> Widget<W>&
    {
        boundedHandler = handler;
        return *this;
    }

private:
    std::optional<Handler> boundedHandler;
};
```

```
            wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");

            btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
                wxLogMessage("Hello world from wxWidgets!");
            });
```

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(bindHandler(new W(parent, id, str, position, size)), flags ? *flags : parentFlags);
    }

    using Handler = std::function<void(wxCommandEvent&)>;
    auto bind(Handler handler) -> Widget<W>&
    {
        boundedHandler = handler;
        return *this;
    }

private:
    std::optional<Handler> boundedHandler;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            widget->Bind(wxEVT_BUTTON, *boundedHandler);
        }
        return widget;
    }
};
```

```
            wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");

            btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
                wxLogMessage("Hello world from wxWidgets!");
            });
```

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(bindHandler(new W(parent, id, str, position, size)), flags ? *flags : parentFlags);
    }

    using Handler = std::function<void(wxCommandEvent&)>;
    auto bind(Handler handler) -> Widget<W>&
    {
        boundedHandler = handler;
        return *this;
    }

private:
    std::optional<Handler> boundedHandler;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            widget->Bind(wxEVT_BUTTON, *boundedHandler);
        }
        return widget;
    }
};
```

```
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(bindHandler(new W(parent, id, str, position, size)), flags ? *flags : parentFlags);
    }

    using Handler = std::function<void(wxCommandEvent&)>;
    auto bind(Handler handler) -> Widget<W>&
    {
        boundedHandler = handler;
        return *this;
    }

private:
    std::optional<Handler> boundedHandler;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            widget->Bind(wxEVT_BUTTON, *boundedHandler);
        }
        return widget;
    }
};
```

```
HSizer {
    TextCtrl { wxSizerFlags(1).Expand().Border() }
        .withWidth(64),
    Button { "Right" },
},
```

```cpp
template <typename W>
struct Widget {

    auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
    {
        sizer->Add(bindHandler(new W(parent, id, str, position, size)), flags ? *flags : parentFlags);
    }

    using Handler = std::function<void(wxCommandEvent&)>;
    auto bind(Handler handler) -> Widget<W>&
    {
        boundedHandler = handler;
        return *this;
    }

private:
    std::optional<Handler> boundedHandler;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            widget->Bind(wxEVT_BUTTON, *boundedHandler);
        }
        return widget;
    }
};
```

```cpp
HSizer {
    TextCtrl { wxSizerFlags(1).Expand().Border() }
        .withWidth(64),
    Button { "Right" }.bind([](wxCommandEvent&) {
        wxLogMessage("Hello world from wxWidgets!");
    }),
},
```

```
HSizer {
    TextCtrl { wxSizerFlags(1).Expand().Border() }
        .withWidth(64),
    Button { "Right" }.bind([](wxCommandEvent&) {
        wxLogMessage("Hello world from wxWidgets!");
    }),
},
```

```
std::function<void(wxCommandEvent&)>
```

```
HSizer {
    TextCtrl { wxSizerFlags(1).Expand().Border() }
        .withWidth(64),
    Button { "Right" }.bind([] {
        wxLogMessage("Hello world from wxWidgets!");
    }),
},
```

```
std::function<void()>
```

```
template <typename W>
struct Widget {
    using Handler = std::function<void(wxCommandEvent&)>;
    auto bind(Handler handler) -> Widget<W>&
    {
        boundedHandler = handler;
        return *this;
    }

private:
    std::optional<Handler> boundedHandler;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            widget->Bind(wxEVT_BUTTON, *boundedHandler);
        }
        return widget;
    }
};
```

```cpp
template <typename W>
struct Widget {
    using Handler = std::variant<std::function<void(wxCommandEvent&)>, std::function<void()>>;
    auto bind(Handler handler) -> Widget<W>&
    {
        boundedHandler = handler;
        return *this;
    }

private:
    std::optional<Handler> boundedHandler;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            widget->Bind(wxEVT_BUTTON, *boundedHandler);
        }
        return widget;
    }
};
```

```cpp
template <typename W>
struct Widget {
    using Handler = std::variant<std::function<void(wxCommandEvent&)>, std::function<void()>>;
    auto bind(Handler handler) -> Widget<W>&
    {
        boundedHandler = handler;
        return *this;
    }

private:
    std::optional<Handler> boundedHandler;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            std::visit(
                details::overloaded {
                    [&widget](std::function<void(wxCommandEvent&)> func) {
                        widget->Bind(wxEVT_BUTTON, func);
                    },
                    [&widget](std::function<void()> func) {
                        widget->Bind(wxEVT_BUTTON, [func](wxCommandEvent&) {
                            func();
                        });
                    },
                },
                *boundedHandler);
        }
        return widget;
    }
};
```



```cpp
template <typename W>
struct Widget {
    using Handler = std::variant<std::function<void(wxCommandEvent&)>, std::function<void()>>;
    auto bind(Handler handler) -> Widget<W>&
    {
        boundedHandler = handler;
        return *this;
    }

private:
    std::optional<Handler> boundedHandler;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            std::visit(
                details::overloaded {
                    [&widget](std::function<void(wxCommandEvent&)> func) {
                        widget->Bind(wxEVT_BUTTON, func);
                    },
                    [&widget](std::function<void()> func) {
                        widget->Bind(wxEVT_BUTTON, [func](wxCommandEvent&) {
                            func();
                        });
                    },
                },
                *boundedHandler);
        }
        return widget;
    }
};
```

???

```cpp
namespace DeclarativeUI::details {

template <class... Ts>
struct overloaded : Ts... {
    using Ts::operator()...;
};
template <class... Ts>
overloaded(Ts...) -> overloaded<Ts...>;

}
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```cpp
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
        wxLogMessage("Hello world from wxWidgets!");
    });

    SetSizerAndFit(sizer);
}
```



```cpp
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");
    wxSlider* slider = new wxSlider(this, wxID_ANY, 3, 1, 10);

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

    sizer->Add(slider, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
        wxLogMessage("Hello world from wxWidgets!");
    });

    SetSizerAndFit(sizer);
}
```



```cpp
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");
    wxSlider* slider = new wxSlider(this, wxID_ANY, 3, 1, 10);

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

    sizer->Add(slider, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
        wxLogMessage("Hello world from wxWidgets!");
    });

    SetSizerAndFit(sizer);
}
```



```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
using TextCtrl = details::Widget<wxTextCtrl>;
using Button = details::Widget<wxButton>;
using Text = details::Widget<wxStaticText>;
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Slider {},
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
using TextCtrl = details::Widget<wxTextCtrl>;
using Button = details::Widget<wxButton>;
using Text = details::Widget<wxStaticText>;
using Slider = details::Widget<wxSlider>;
```

```
using namespace DeclarativeUI;
// Create and layout the controls
VSizer {
    wxSizerFlags().Expand().Borde
    HSizer {
        TextCtrl { wxSizerFlags(1
            .withWidth(64),
        Button { "Right" }.bind([
            wxLogMessage("Hello w
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Slider {},
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
[main] Building folder: /Users/richardpowell/Development/github.com/rmpowell77/
wxHelloWorld.git/build all
[build] Starting build
[proc] Executing command: /opt/homebrew/bin/cmake --build /Users/richardpowell/
Development/github.com/rmpowell77/wxHelloWorld.git/build --config Debug --target all --
[build] [1/2  50% :: 1.001] Building CXX object CMakeFiles/wxHelloWorld.dir/
HelloWorld.cpp.o
[build] FAILED: CMakeFiles/wxHelloWorld.dir/HelloWorld.cpp.o
[build] /usr/bin/clang++ -DWXUSINGDLL -D_FILE_OFFSET_BITS=64 -D__WXMAC__
-D__WXOSX_COCOA__ -D__WXOSX__ -isystem /opt/homebrew/lib/wx/include/osx_cocoa-
unicode-3.2 -isystem /opt/homebrew/include/wx-3.2 -g -std=c++20 -arch arm64 -isysroot /
Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/
MacOSX15.2.sdk -Wall -Wextra -Werror -MD -MT CMakeFiles/wxHelloWorld.dir/
HelloWorld.cpp.o -MF CMakeFiles/wxHelloWorld.dir/HelloWorld.cpp.o.d -o CMakeFiles/
wxHelloWorld.dir/HelloWorld.cpp.o -c /Users/richardpowell/Development/github.com/
rmpowell77/wxHelloWorld.git/HelloWorld.cpp
[build] /Users/richardpowell/Development/github.com/rmpowell77/wxHelloWorld.git/
HelloWorld.cpp:50:24: error: no matching constructor for initialization of 'wxSlider'
[build]    50 |         sizer->Add(new W(parent, id, str, wxDefaultPosition, size),
flags ? *flags : parentFlags);
[build]       |                        ^ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
[build] /Users/richardpowell/Development/github.com/rmpowell77/wxHelloWorld.git/
HelloWorld.cpp:13:12: note: in instantiation of member function
'DeclarativeUI::details::Widget<wxSlider>::createAndAdd' requested here
[build]    13 |     widget.createAndAdd(window, sizer, wxSizerFlags {});
[build]       |            ^
[build] /Users/richardpowell/Development/github.com/rmpowell77/wxHelloWorld.git/
HelloWorld.cpp:13:5: note: in instantiation of requirement here
[build]    13 |     widget.createAndAdd(window, sizer, wxSizerFlags {});
[build]       |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
[build] /Users/richardpowell/Development/github.com/rmpowell77/wxHelloWorld.git/
HelloWorld.cpp:12:28: note: while substituting template arguments into constraint
expression here
[build]    12 | concept CreateAndAddable = requires(T widget, wxWindow* window, wxSizer*
sizer) {
[build]       |
^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
[build]    13 |     widget.createAndAdd(window, sizer, wxSizerFlags {});
[build]       |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
[build]    14 | };
[build]       | ~
[build] /Users/richardpowell/Development/github.com/rmpowell77/wxHelloWorld.git/
HelloWorld.cpp:138:11: note: while checking the satisfaction of concept
'CreateAndAddable<DeclarativeUI::details::Widget<wxSlider>>' requested here
```

wxTextCtrl() [2/2]

```
wxTextCtrl::wxTextCtrl(wxWindow *          parent,
                       wxWindowID          id,
                       const wxString &    value = wxEmptyString,
                       const wxPoint &     pos = wxDefaultPosition,
                       const wxSize &      size = wxDefaultSize,
                       long                style = 0,
                       const wxValidator & validator = wxDefaultValidator,
                       const wxString &    name = wxTextCtrlNameStr)
```

wxStaticText() [2/2]

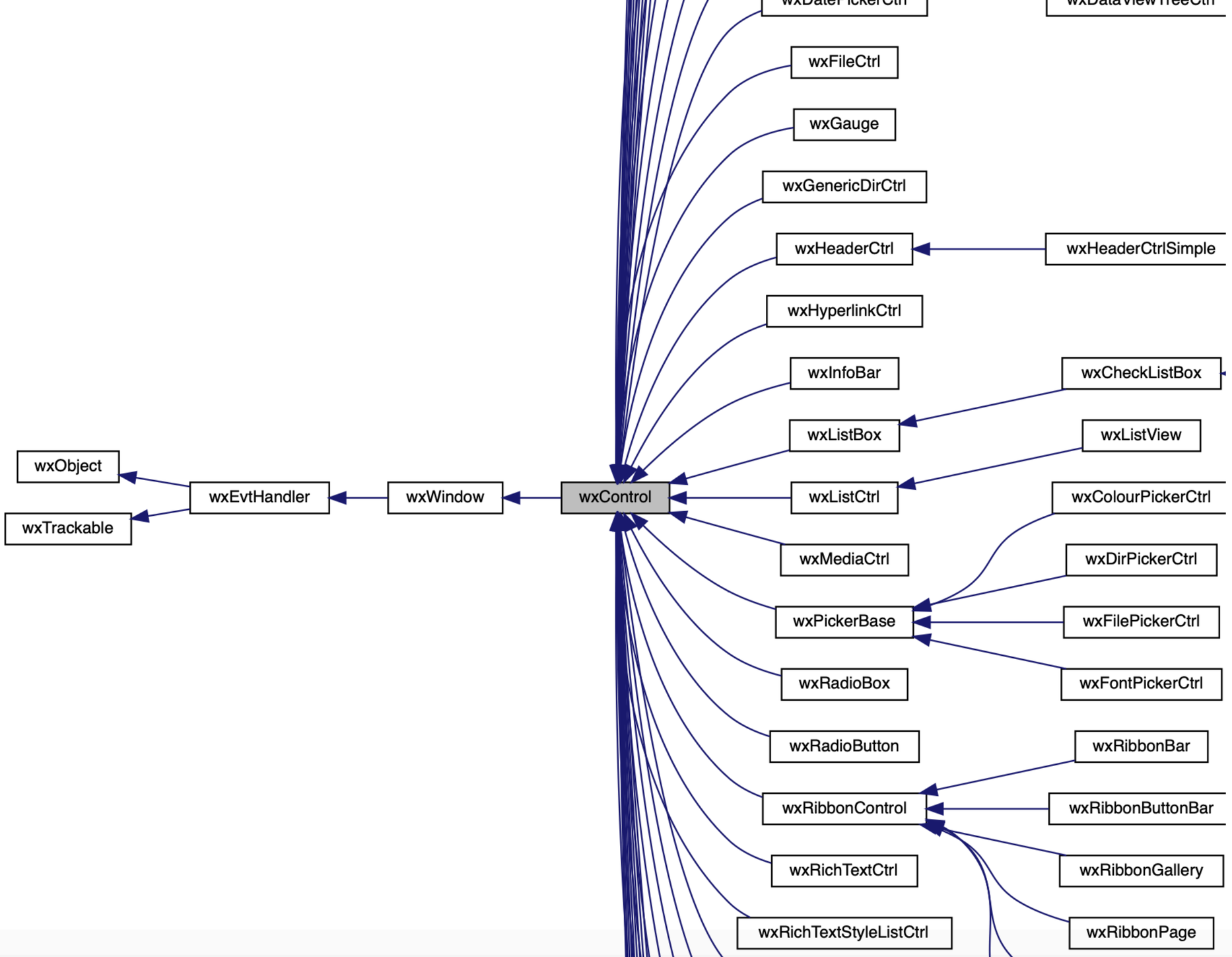
```
wxStaticText::wxStaticText(wxWindow *       parent,
                           wxWindowID       id,
                           const wxString & label,
                           const wxPoint &  pos = wxDefaultPosition,
                           const wxSize &   size = wxDefaultSize,
                           long             style = 0,
                           const wxString & name = wxStaticTextNameStr)
```

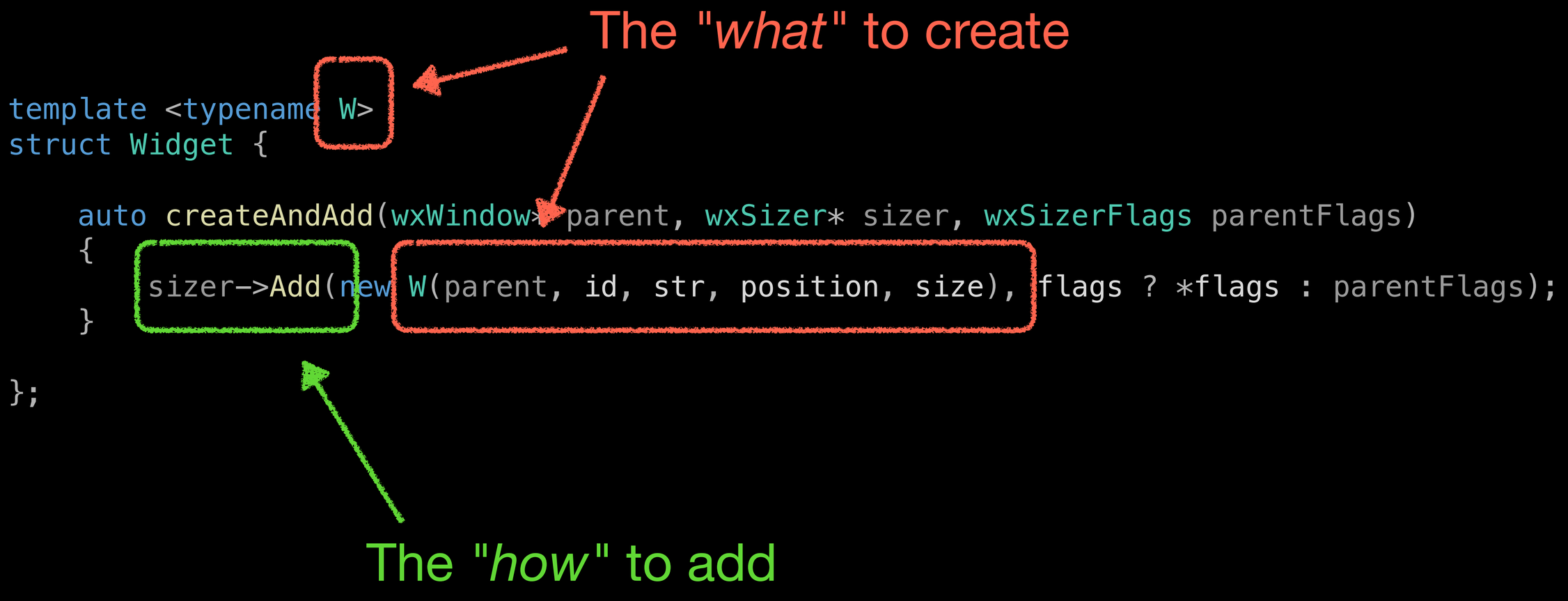
wxButton() [2/2]

```
wxButton::wxButton(wxWindow *          parent,
                   wxWindowID          id,
                   const wxString &    label = wxEmptyString,
                   const wxPoint &     pos = wxDefaultPosition,
                   const wxSize &      size = wxDefaultSize,
                   long                style = 0,
                   const wxValidator & validator = wxDefaultValidator,
                   const wxString &    name = wxButtonNameStr)
```

wxSlider() [2/2]

```
wxSlider::wxSlider(wxWindow *          parent,
                   wxWindowID          id,
                   int                 value,
                   int                 minValue,
                   int                 maxValue,
                   const wxPoint &     pos = wxDefaultPosition,
                   const wxSize &      size = wxDefaultSize,
                   long                style = wxSL_HORIZONTAL,
                   const wxValidator & validator = wxDefaultValidator,
                   const wxString &    name = wxSliderNameStr)
```


wxObject
wxTrackable
wxEvtHandler
wxWindow
wxControl
wxFileCtrl
wxGauge
wxGenericDirCtrl
wxHeaderCtrl
wxHeaderCtrlSimple
wxHyperlinkCtrl
wxInfoBar
wxCheckListBox
wxListBox
wxListView
wxListCtrl
wxMediaCtrl
wxColourPickerCtrl
wxDirPickerCtrl
wxPickerBase
wxFilePickerCtrl
wxFontPickerCtrl
wxRadioBox
wxRadioButton
wxRibbonBar
wxRibbonControl
wxRibbonButtonBar
wxRichTextCtrl
wxRibbonGallery
wxRichTextStyleListCtrl
wxRibbonPage

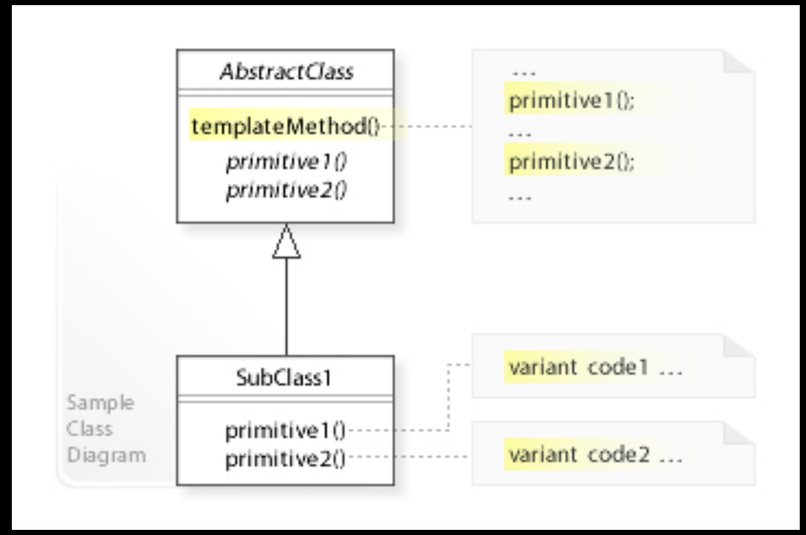

The "*what*" to create
template <typename W>
struct Widget {
auto createAndAdd(wxWindow* parent, wxSizer* sizer, wxSizerFlags parentFlags)
{
sizer->Add(new W(parent, id, str, position, size), flags ? *flags : parentFlags);
}
};
The "*how*" to add

Template method pattern

The ***template method*** is a method in a superclass, and defines the skeleton of an operation in terms of a number of high-level steps.

These steps are implemented by additional *helper methods*.

Subclasses customize the operation by overriding the *helper methods*.



https://en.wikipedia.org/wiki/Template_method_pattern

```
template <typename W>
struct Widget {
    explicit Widget(
        wxWindowID id = wxID_ANY,
        std::string str = std::string {},
        std::optional<wxSizerFlags> flags = {})
        : id(id)
        , str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(
        wxWindow* parent,
        wxSizer* sizer,
        wxSizerFlags parentFlags)
    {
        sizer->Add(
            new W(parent, id, str, position, size),
            flags ? *flags : parentFlags);
    }

private:
    wxWindowID id;
};
```

```
template <typename W>
struct Widget {
    explicit Widget(
        wxWindowID id = wxID_ANY,
        std::string str = std::string {},
        std::optional<wxSizerFlags> flags = {})
        : id(id)
        , str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(
        wxWindow* parent,
        wxSizer* sizer,
        wxSizerFlags parentFlags)
    {
        sizer->Add(
            new W(parent, id, str, position, size),
            flags ? *flags : parentFlags);
    }

private:
    virtual auto create(
        wxWindow* parent,
        wxWindowID id,
        std::string const& str,
        wxPoint pos,
        wxSize size) -> wxWindow* = 0;

    wxWindowID id;
};
```

```
template <typename W>
struct Widget {
    explicit Widget(
        wxWindowID id = wxID_ANY,
        std::string str = std::string {},
        std::optional<wxSizerFlags> flags = {})
        : id(id)
        , str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(
        wxWindow* parent,
        wxSizer* sizer,
        wxSizerFlags parentFlags)
    {
        sizer->Add(
            new W(parent, id, str, position, size),
            flags ? *flags : parentFlags);
    }

private:
    virtual auto create(
        wxWindow* parent,
        wxWindowID id,
        std::string const& str,
        wxPoint pos,
        wxSize size) -> wxWindow* = 0;

    wxWindowID id;
};
```

```
template <typename W>
struct Widget {
    explicit Widget(
        wxWindowID id = wxID_ANY,
        std::string str = std::string {},
        std::optional<wxSizerFlags> flags = {})
        : id(id)
        , str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(
        wxWindow* parent,
        wxSizer* sizer,
        wxSizerFlags parentFlags)
    {
        sizer->Add(
            create(parent, id, str, position, size),
            flags ? *flags : parentFlags);
    }

private:
    virtual auto create(
        wxWindow* parent,
        wxWindowID id,
        std::string const& str,
        wxPoint pos,
        wxSize size) -> wxWindow* = 0;

    wxWindowID id;
};
```

```cpp
struct Widget {
    explicit Widget(
        wxWindowID id = wxID_ANY,
        std::string str = std::string {},
        std::optional<wxSizerFlags> flags = {})
        : id(id)
        , str(std::move(str))
        , flags(flags)
    {
    }

    auto createAndAdd(
        wxWindow* parent,
        wxSizer* sizer,
        wxSizerFlags parentFlags)
    {
        sizer->Add(
            create(parent, id, str, position, size),
            flags ? *flags : parentFlags);
    }

private:
    virtual auto create(
        wxWindow* parent,
        wxWindowID id,
        std::string const& str,
        wxPoint pos,
        wxSize size) -> wxWindow* = 0;

    wxWindowID id;
};
```

The "*what*" to create

The "*how*" to add

```cpp
struct Text : details::Widget {
    using super = details::Widget;
    explicit Text(
        wxWindowID id = wxID_ANY,
        std::string str = std::string {},
        std::optional<wxSizerFlags> flags = {})
        : super(id, std::move(str), flags)
    {
    }

private:
    auto create(
        wxWindow* parent,
        wxWindowID id,
        std::string const& str,
        wxPoint pos,
        wxSize size) -> wxWindow*
    {
        return new wxStaticText(parent, id, str, pos
    }
};
```

```
using namespace DeclarativeUI;
// Create and layout the controls
VSizer {
    wxSizerFlags().Expand().Borde
    HSizer {
        TextCtrl { wxSizerFlags(1
            .withWidth(64),
        Button { "Right" }.bind([
            wxLogMessage("Hello w
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
[main] Building folder: /Users/richardpowell/Development/github.com/rmpowell77/
wxHelloWorld.git/build
[build] Starting build
[proc] Executing command: /opt/homebrew/bin/cmake --build /Users/richardpowell/
Development/github.com/rmpowell77/wxHelloWorld.git/build --config Debug --target all --
[build] [1/2  50% :: 0.927] Building CXX object CMakeFiles/wxHelloWorld.dir/
HelloWorld.cpp.o
[build] FAILED: CMakeFiles/wxHelloWorld.dir/HelloWorld.cpp.o
[build] /usr/bin/clang++ -DWXUSINGDLL -D_FILE_OFFSET_BITS=64 -D__WXMAC__
-D__WXOSX_COCOA__ -D__WXOSX__ -isystem /opt/homebrew/lib/wx/include/osx_cocoa-
unicode-3.2 -isystem /opt/homebrew/include/wx-3.2 -g -std=c++20 -arch arm64 -isysroot /
Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/
MacOSX15.2.sdk -Wall -Wextra -Werror -MD -MT CMakeFiles/wxHelloWorld.dir/
HelloWorld.cpp.o -MF CMakeFiles/wxHelloWorld.dir/HelloWorld.cpp.o.d -o CMakeFiles/
wxHelloWorld.dir/HelloWorld.cpp.o -c /Users/richardpowell/Development/github.com/
rmpowell77/wxHelloWorld.git/HelloWorld.cpp
[build] In file included from /Users/richardpowell/Development/github.com/rmpowell77/
wxHelloWorld.git/HelloWorld.cpp:5:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/wx.h:15:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/object.h:19:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/memory.h:15:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/string.h:37:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/strvararg.h:19:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/unichar.h:15:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/stringimpl.h:66:
[build] In file included from /Applications/Xcode.app/Contents/Developer/Platforms/
MacOSX.platform/Developer/SDKs/MacOSX15.2.sdk/usr/include/c++/v1/string:594:
[build] In file included from /Applications/Xcode.app/Contents/Developer/Platforms/
MacOSX.platform/Developer/SDKs/MacOSX15.2.sdk/usr/include/c++/v1/__memory_resource/
polymorphic_allocator.h:20:
[build] /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/
SDKs/MacOSX15.2.sdk/usr/include/c++/v1/tuple:306:7: error: field type
'DeclarativeUI::details::Widget' is an abstract class
[build]   306 |   _Hp __value_;
[build]       |       ^
[build] /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/
SDKs/MacOSX15.2.sdk/usr/include/c++/v1/tuple:464:14: note: in instantiation of template
class 'std::__tuple_leaf<0, DeclarativeUI::details::Widget>' requested here
[build]   464 |     : public __tuple_leaf<_Indx, _Tp>... {
[build]       |              ^
[build] /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/
SDKs/MacOSX15.2.sdk/usr/include/c++/v1/tuple:536:10: note: in instantiation of template
class 'std::__tuple_impl<std::__tuple_indices<0, 1>, DeclarativeUI::details::Widget,
DeclarativeUI::Button>' requested here
```

```
using namespace DeclarativeUI;
// Create and layout the controls
VSizer {
    wxSizerFlags().Expand().Borde
    HSizer {
        TextCtrl { wxSizerFlags(1
            .withWidth(64),
        Button { "Right" }.bind([
            wxLogMessage("Hello w
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
[main] Building folder: /Users/richardpowell/Development/github.com/rmpowell77/
wxHelloWorld.git/build
[build] Starting build
[proc] Executing command: /opt/homebrew/bin/cmake --build /Users/richardpowell/
Development/github.com/rmpowell77/wxHelloWorld.git/build --config Debug --target all --
[build] [1/2  50% :: 0.927] Building CXX object CMakeFiles/wxHelloWorld.dir/
HelloWorld.cpp.o
[build] FAILED: CMakeFiles/wxHelloWorld.dir/HelloWorld.cpp.o
[build] /usr/bin/clang++ -DWXUSINGDLL -D_FILE_OFFSET_BITS=64 -D__WXMAC__
-D__WXOSX_COCOA__ -D__WXOSX__ -isystem /opt/homebrew/lib/wx/include/osx_cocoa-
unicode-3.2 -isystem /opt/homebrew/include/wx-3.2 -g -std=c++20 -arch arm64 -isysroot /
Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/
MacOSX15.2.sdk -Wall -Wextra -Werror -MD -MT CMakeFiles/wxHelloWorld.dir/
HelloWorld.cpp.o -MF CMakeFiles/wxHelloWorld.dir/HelloWorld.cpp.o.d -o CMakeFiles/
wxHelloWorld.dir/HelloWorld.cpp.o -c /Users/richardpowell/Development/github.com/
rmpowell77/wxHelloWorld.git/HelloWorld.cpp
[build] In file included from /Users/richardpowell/Development/github.com/rmpowell77/
wxHelloWorld.git/HelloWorld.cpp:5:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/wx.h:15:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/object.h:19:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/memory.h:15:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/string.h:37:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/strvararg.h:19:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/unichar.h:15:
[build] In file included from /opt/homebrew/include/wx-3.2/wx/stringimpl.h:66:
[build] In file included from /Applications/Xcode.app/Contents/Developer/Platforms/
MacOSX.platform/Developer/SDKs/MacOSX15.2.sdk/usr/include/c++/v1/string:594:
[build] In file included from /Applications/Xcode.app/Contents/Developer/Platforms/
MacOSX.platform/Developer/SDKs/MacOSX15.2.sdk/usr/include/c++/v1/__memory_resource/
polymorphic_allocator.h:20:
[build] /Applications/Xcode.app/Contents/Developer/Platforms/
MacOSX.platform/Developer/SDKs/MacOSX15.2.sdk/usr/include/c++/v1/
tuple:306:7: error: field type 'DeclarativeUI::details::Widget'
is an abstract class
[build]   306 |   _Hp __value_;
[build]       |       ^
[build] /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/
SDKs/MacOSX15.2.sdk/usr/include/c++/v1/tuple:464:14: note: in instantiation of template
class 'std::__tuple_leaf<0, DeclarativeUI::details::Widget>' requested here
[build]   464 |     : public __tuple_leaf<_Indx, _Tp>... {
[build]       |              ^
[build] /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/
SDKs/MacOSX15.2.sdk/usr/include/c++/v1/tuple:536:10: note: in instantiation of template
```

```
struct Widget {

    auto withSize(wxSize size_) -> Widget&
    {
        size = size_;
        return *this;
    }

    auto withWidth(int size_) -> Widget&
    {
        size.SetWidth(size_);
        return *this;
    }

    auto withHeight(int size_) -> Widget&
    {
        size.SetHeight(size_);
        return *this;
    }

private:
    virtual auto create(
        wxWindow* parent,
        wxWindowID id,
        std::string const& str,
        wxPoint pos,
        wxSize size) -> wxWindow* = 0;
};
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!"
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
struct Widget {

    auto withSize(wxSize size_) -> TextCtrl&
    {
        size = size_;
        return *this;
    }

    auto withWidth(int size_) -> TextCtrl&
    {
        size.SetWidth(size_);
        return *this;
    }

    auto withHeight(int size_) -> TextCtrl&
    {
        size.SetHeight(size_);
        return *this;
    }

private:
    virtual auto create(
        wxWindow* parent,
        wxWindowID id,
        std::string const& str,
        wxPoint pos,
        wxSize size) -> wxWindow* = 0;
};
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!"
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

struct TextCtrl : details::Widget;

```
struct Widget {

    auto withSize(wxSize size_) -> TextCtrl&
    {
        size = size_;
        return *this;
    }

    auto withWidth(int size_) -> TextCtrl&
    {
        size.SetWidth(size_);
        return *this;
    }

    auto withHeight(int size_) -> TextCtrl&
    {
        size.SetHeight(size_);
        return *this;
    }

private:
    virtual auto create(
        wxWindow* parent,
        wxWindowID id,
        std::string const& str,
        wxPoint pos,
        wxSize size) -> wxWindow* = 0;
};
```

struct Button : details::Widget;

```
struct Widget {

    auto withSize(wxSize size_) -> Button&
    {
        size = size_;
        return *this;
    }

    auto withWidth(int size_) -> Button&
    {
        size.SetWidth(size_);
        return *this;
    }

    auto withHeight(int size_) -> Button&
    {
        size.SetHeight(size_);
        return *this;
    }

private:
    virtual auto create(
        wxWindow* parent,
        wxWindowID id,
        std::string const& str,
        wxPoint pos,
        wxSize size) -> wxWindow* = 0;
};
```



struct Button : details::Widget;

```
template <typename W>
struct Widget {

    auto withSize(wxSize size_) -> Button&
    {
        size = size_;
        return *this;
    }

    auto withWidth(int size_) -> Button&
    {
        size.SetWidth(size_);
        return *this;
    }

    auto withHeight(int size_) -> Button&
    {
        size.SetHeight(size_);
        return *this;
    }

private:
    virtual auto create(
        wxWindow* parent,
        wxWindowID id,
        std::string const& str,
        wxPoint pos,
        wxSize size) -> wxWindow* = 0;
};
```

struct Button : details::Widget<Button>;

```
template <typename W>
struct Widget {

    auto withSize(wxSize size_) -> W&
    {
        size = size_;
        return static_cast<W&>(*this);
    }

    auto withWidth(int size_) -> W&
    {
        size.SetWidth(size_);
        return static_cast<W&>(*this);
    }

    auto withHeight(int size_) -> W&
    {
        size.SetHeight(size_);
        return static_cast<W&>(*this);
    }

private:
    virtual auto create(
        wxWindow* parent,
        wxWindowID id,
        std::string const& str,
        wxPoint pos,
        wxSize size) -> wxWindow* = 0;
};
```

```
struct Button : details::Widget<Button>;
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
using TextCtrl = details::Widget<wxTextCtrl>;
using Button = details::Widget<wxButton>;
using Text = details::Widget<wxStaticText>;
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
struct TextCtrl : details::Widget<TextCtrl>;
struct Button : details::Widget<Button>;
struct Text : details::Widget<Text>;
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
struct TextCtrl : details::Widget<TextCtrl>;
struct Button : details::Widget<Button>;
struct Text : details::Widget<Text>;
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Slider {},
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

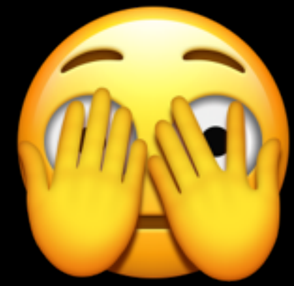



```
struct TextCtrl : details::Widget<TextCtrl>;
struct Button : details::Widget<Button>;
struct Text : details::Widget<Text>;
struct Slider : details::Widget<Slider>;
```

◆ wxSlider() [2/2]

```
wxSlider::wxSlider ( wxWindow *          parent,
                     wxWindowID          id,
                     int                 value,
                     int                 minValue,
                     int                 maxValue,
                     const wxPoint &     pos =  wxDefaultPosition ,
                     const wxSize &      size =  wxDefaultSize ,
                     long                style =  wxSL_HORIZONTAL ,
                     const wxValidator & validator =  wxDefaultValidator ,
                     const wxString &    name =  wxSliderNameStr
                   )
```

```
typedef int wxWindowID;
```



◆ wxSlider() [2/2]

```
wxSlider::wxSlider ( wxWindow *          parent,
                     wxWindowID          id,
                     int                 value,
                     int                 minValue,
                     int                 maxValue,
                     const wxPoint &     pos =  wxDefaultPosition ,
                     const wxSize &      size =  wxDefaultSize ,
                     long                style =  wxSL_HORIZONTAL ,
                     const wxValidator & validator =  wxDefaultValidator ,
                     const wxString &    name =  wxSliderNameStr
                   )
```

```
struct Slider : details::Widget<Slider> {
    using super = details::Widget<Slider>;
    explicit Slider(std::pair<int, int> range, std::optional<wxSizerFlags> flags = {})
        : Slider(wxID_ANY, range, range.first, flags)
    {
    }

    explicit Slider(std::pair<int, int> range, int value, std::optional<wxSizerFlags> flags = {})
        : Slider(wxID_ANY, range, value, flags)
    {
    }

    explicit Slider(wxWindowID id, std::pair<int, int> range, std::optional<wxSizerFlags> flags = {})
        : Slider(id, range, range.first, flags)
    {
    }

    explicit Slider(wxWindowID id, std::pair<int, int> range, int value, std::optional<wxSizerFlags> flags = {})
        : super(id, std::string {}, flags)
        , range(range)
        , value(value)
    {
    }

private:
    auto create(wxWindow* parent, wxWindowID id, std::string const&, wxPoint pos, wxSize size) -> wxWindow*
    {
        return new wxSlider(parent, id, value, range.first, range.second, pos, size);
    }
    std::pair<int, int> range;
    int value;
};
```

```cpp
template <typename W>
struct Widget {
private:
    wxWindowID id;
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
    std::string str;
    std::optional<wxSizerFlags> flags;
    std::optional<Handler> boundedHandler;
};
```

```cpp
struct Text : details::Widget<Text> {
private:

};
```

```cpp
struct TextCtrl : details::Widget<TextCtrl> {
private:

};
```

```cpp
struct Button : details::Widget<Button> {
private:

};
```

```cpp
struct Slider : details::Widget<Slider> {
private:
    std::pair<int, int> range;
    int value;
};
```

```cpp
template <typename W>
struct Widget {
private:
    wxWindowID id;
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
    std::optional<wxSizerFlags> flags;
    std::optional<Handler> boundedHandler;
};
```

```cpp
struct Text : details::Widget<Text> {
private:
    std::string str;
};
```

```cpp
struct TextCtrl : details::Widget<TextCtrl> {
private:
    std::string str;
};
```

```cpp
struct Button : details::Widget<Button> {
private:
    std::string str;
};
```

```cpp
struct Slider : details::Widget<Slider> {
private:
    std::pair<int, int> range;
    int value;
};
```

```cpp
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");
    wxSlider* slider = new wxSlider(this, wxID_ANY, 3, 1, 10);

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

    sizer->Add(slider, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
        wxLogMessage("Hello world from wxWidgets!");
    });

    SetSizerAndFit(sizer);
}
```



```cpp
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    CreateStatusBar(1);
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");
    wxSlider* slider = new wxSlider(this, wxID_ANY, 3, 1, 10);

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

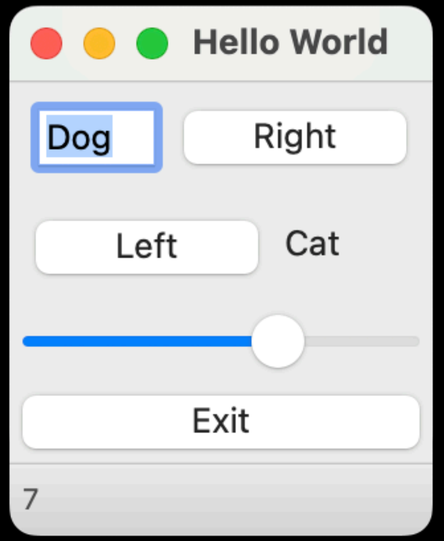
    sizer->Add(slider, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
        wxLogMessage("Hello world from wxWidgets!");
    });

    slider->Bind(wxEVT_SLIDER, [this](wxCommandEvent& event) {
        SetStatusText(std::to_string(event.GetInt()));
    });

    SetSizerAndFit(sizer);
}
```



```cpp
MyFrame::MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size)
    : wxFrame(NULL, wxID_ANY, title, pos, size)
{
    CreateStatusBar(1);
    // Create the controls.
    wxTextCtrl* text1 = new wxTextCtrl(this, wxID_ANY, "Dog");
    wxButton* btnRight = new wxButton(this, wxID_ANY, "Right");
    wxButton* btnLeft = new wxButton(this, wxID_ANY, "Left");
    wxStaticText* text2 = new wxStaticText(this, wxID_ANY, "Cat");
    wxButton* btnExit = new wxButton(this, wxID_EXIT, "Exit");
    wxSlider* slider = new wxSlider(this, wxID_ANY, 3, 1, 10);

    // Layout the controls.
    wxBoxSizer* sizer = new wxBoxSizer(wxVERTICAL);

    wxBoxSizer* sizerText = new wxBoxSizer(wxHORIZONTAL);
    sizerText->Add(text1, wxSizerFlags(1).Expand().Border());
    sizerText->Add(btnRight, wxSizerFlags().Expand().Border());

    sizer->Add(sizerText, wxSizerFlags().Expand().Border());

    wxBoxSizer* sizerBtns = new wxBoxSizer(wxHORIZONTAL);
    sizerBtns->Add(btnLeft, wxSizerFlags().Expand().Border());
    sizerBtns->Add(text2, wxSizerFlags().Expand().Border());

    sizer->Add(sizerBtns, wxSizerFlags().Expand().Border());

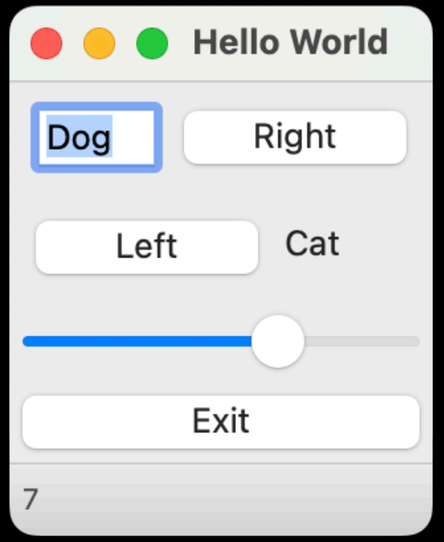
    sizer->Add(slider, wxSizerFlags().Expand().Border());

    sizer->Add(btnExit, wxSizerFlags().Expand().Border());

    btnRight->Bind(wxEVT_BUTTON, [](wxCommandEvent&) {
        wxLogMessage("Hello world from wxWidgets!");
    });

    slider->Bind(wxEVT_SLIDER, [this](wxCommandEvent& event) {
        SetStatusText(std::to_string(event.GetInt()));
    });

    SetSizerAndFit(sizer);
}
```

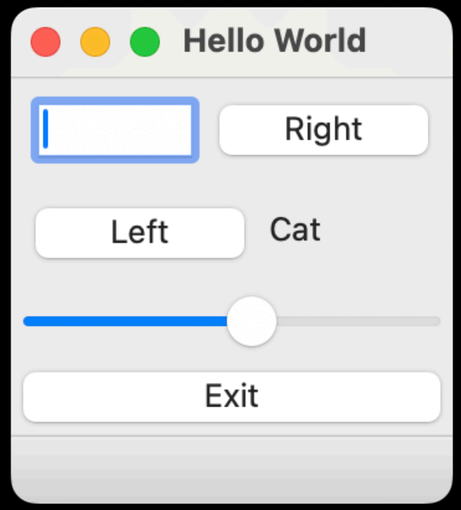



```cpp
slider->Bind(wxEVT_SLIDER, [this](wxCommandEvent& event) {
    SetStatusText(std::to_string(event.GetInt()));
});
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Slider { { 1, 10 }, 3 },
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Slider { { 1, 10 }, 3 },bind([this](wxCommandEvent& event) {
        SetStatusText(std::to_string(event.GetInt()));
    }),
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

```cpp
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Slider { { 1, 10 }, 3 }.bind([this](wxCommandEvent& event) {
        SetStatusText(std::to_string(event.GetInt()));
    }),
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```



```cpp
template <typename W>
struct Widget {
    using Handler = std::variant<std::function<void(wxCommandEvent&)>, std::function<void()>>;
    auto bind(Handler handler) -> W&
    {
        boundedHandler = handler;
        return static_cast<W&>(*this);
    }

private:
    std::optional<Handler> boundedHandler;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            std::visit(
                details::overloaded {
                    [&widget](std::function<void(wxCommandEvent&)> func) {
                        widget->Bind(wxEVT_BUTTON, func);
                    },
                    [&widget](std::function<void()> func) {
                        widget->Bind(wxEVT_BUTTON, [func](wxCommandEvent&) {
                            func();
                        });
                    },
                },
                *boundedHandler);
        }
        return widget;
    }
};
```



```cpp
template <typename W>
struct Widget {
    using Handler = std::variant<std::function<void(wxCommandEvent&)>, std::function<void()>>;
    auto bind(Handler handler) -> W&
    {
        boundedHandler = handler;
        return static_cast<W&>(*this);
    }

private:
    std::optional<Handler> boundedHandler;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            std::visit(
                details::overloaded {
                    [&widget](std::function<void(wxCommandEvent&)> func) {
                        widget->Bind(wxEVT_BUTTON, func);
                    },
                    [&widget](std::function<void()> func) {
                        widget->Bind(wxEVT_BUTTON, [func](wxCommandEvent&) {
                            func();
                        });
                    },
                },
                *boundedHandler);
        }
        return widget;
    }
};
```

```cpp
template <typename W>
struct Widget {
    using Handler = std::variant<std::function<void(wxCommandEvent&)>, std::function<void()>>;
    auto bind(wxEventTypeTag<wxCommandEvent> event, Handler handler) -> W&
    {
        boundedHandlers[event] = handler;
        return static_cast<W&>(*this);
    }

private:
    std::map<wxEventTypeTag<wxCommandEvent>, Handler> boundedHandlers;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            std::visit(
                details::overloaded {
                    [&widget](std::function<void(wxCommandEvent&)> func) {
                        widget->Bind(wxEVT_BUTTON, func);
                    },
                    [&widget](std::function<void()> func) {
                        widget->Bind(wxEVT_BUTTON, [func](wxCommandEvent&) {
                            func();
                        });
                    },
                },
                *boundedHandler);
        }
        return widget;
    }
};
```

```cpp
template <typename W>
struct Widget {
    using Handler = std::variant<std::function<void(wxCommandEvent&)>, std::function<void()>>;
    auto bind(wxEventTypeTag<wxCommandEvent> event, Handler handler) -> W&
    {
        boundedHandlers[event] = handler;
        return static_cast<W&>(*this);
    }

private:
    std::map<wxEventTypeTag<wxCommandEvent>, Handler> boundedHandlers;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        if (boundedHandler) {
            std::visit(
                details::overloaded {
                    [&widget](std::function<void(wxCommandEvent&)> func) {
                        widget->Bind(wxEVT_BUTTON, func);
                    },
                    [&widget](std::function<void()> func) {
                        widget->Bind(wxEVT_BUTTON, [func](wxCommandEvent&) {
                            func();
                        });
                    },
                },
                *boundedHandler);
        }
        return widget;
    }
};
```

```cpp
template <typename W>
struct Widget {
    using Handler = std::variant<std::function<void(wxCommandEvent&)>, std::function<void()>>;
    auto bind(wxEventTypeTag<wxCommandEvent> event, Handler handler) -> W&
    {
        boundedHandlers[event] = handler;
        return static_cast<W&>(*this);
    }

private:
    std::map<wxEventTypeTag<wxCommandEvent>, Handler> boundedHandlers;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        for (auto&& [event, func] : boundedHandlers) {
            std::visit(
                details::overloaded {
                    [&widget, &event](std::function<void(wxCommandEvent&)> func) {
                        widget->Bind(event, func);
                    },
                    [&widget, &event](std::function<void()> func) {
                        widget->Bind(event, [func](wxCommandEvent&) {
                            func();
                        });
                    },
                },
                func);
        }
        return widget;
    }
};
```

```cpp
template <typename W>
struct Widget {
    using Handler = std::variant<std::function<void(wxCommandEvent&)>, std::function<void()>>;
    auto bind(wxEventTypeTag<wxCommandEvent> event, Handler handler) -> W&
    {
        boundedHandlers[event] = handler;
        return static_cast<W&>(*this);
    }

private:
    std::map<wxEventTypeTag<wxCommandEvent>, Handler> boundedHandlers;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        for (auto&& [event, func] : boundedHandlers) {
            std::visit(
                details::overloaded {
                    [&widget, &event](std::function<void(wxCommandEvent&)> func) {
                        widget->Bind(event, func);
                    },
                    [&widget, &event](std::function<void()> func) {
                        widget->Bind(event, [func](wxCommandEvent&) {
                            func();
                        });
                    },
                },
                func);
        }
        return widget;
    }
};
```

```cpp
template <typename W>
struct Widget {
    using Handler = std::variant<std::function<void(wxCommandEvent&)>, std::function<void()>>;
protected:
    auto bind(wxEventTypeTag<wxCommandEvent> event, Handler handler) -> W&
    {
        boundedHandlers[event] = handler;
        return static_cast<W&>(*this);
    }

private:
    std::map<wxEventTypeTag<wxCommandEvent>, Handler> boundedHandlers;
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        for (auto&& [event, func] : boundedHandlers) {
            std::visit(
                details::overloaded {
                    [&widget, &event](std::function<void(wxCommandEvent&)> func) {
                        widget->Bind(event, func);
                    },
                    [&widget, &event](std::function<void()> func) {
                        widget->Bind(event, [func](wxCommandEvent&) {
                            func();
                        });
                    },
                },
                func);
        }
        return widget;
    }
};
```

```cpp
template <typename W>
struct Widget {
    using Handler = std::variant<std::function<void(wxCommandEvent&)>
protected:
    auto bind(wxEventTypeTag<wxCommandEvent> event, Handler handler)
    {
        boundedHandlers[event] = handler;
        return static_cast<W&>(*this);
    }

private:
    std::map<wxEventTypeTag<wxCommandEvent>, Handler> boundedHandlers
    auto bindHandler(wxWindow* widget) -> wxWindow*
    {
        for (auto&& [event, func] : boundedHandlers) {
            std::visit(
                details::overloaded {
                    [&widget, &event](std::function<void(wxCommandEve
                        widget->Bind(event, func);
                    },
                    [&widget, &event](std::function<void()> func) {
                        widget->Bind(event, [func](wxCommandEvent&)
                            func();
                        });
                    },
                },
                func);
        }
        return widget;
    }
};
```

```cpp
struct Button : details::Widget<Button> {

    auto bind(Handler handler) -> Button&
    {
        return super::bind(wxEVT_BUTTON, handler);
    }

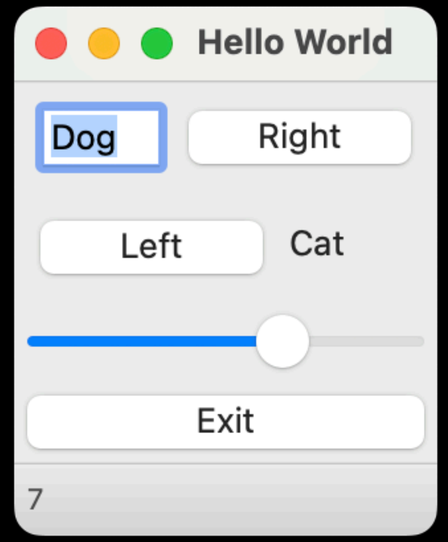
};
```

```cpp
struct Slider : details::Widget<Slider> {

    auto bind(Handler handler) -> Slider&
    {
        return super::bind(wxEVT_SLIDER, handler);
    }

};
```

```cpp
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Slider { { 1, 10 }, 3 }.bind([this](wxCommandEvent& event) {
        SetStatusText(std::to_string(event.GetInt()));
    }),
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```



Hide complexity

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        Widget<wxTextCtrl> { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Widget<wxButton> { "Right" },
    },
    HSizer {
        Widget<wxButton> { "Left" },
        Widget<wxStaticText> { "Cat" },
    },
}
    .attachTo(this);
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Slider { { 1, 10 }, 3 }.bind([this](wxCommandEvent& event) {
        SetStatusText(std::to_string(event.GetInt()));
    }),
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

Allowed for Declarative building

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        Widget<wxTextCtrl> { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Widget<wxButton> { "Right" },
    },
    HSizer {
        Widget<wxButton> { "Left" },
        Widget<wxStaticText> { "Cat" },
    },
}
    .attachTo(this);
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Slider { { 1, 10 }, 3 }.bind([this](wxCommandEvent& event) {
        SetStatusText(std::to_string(event.GetInt()));
    }),
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

Anticipated different users needs

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        Widget<wxTextCtrl> { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Widget<wxButton> { "Right" },
    },
    HSizer {
        Widget<wxButton> { "Left" },
        Widget<wxStaticText> { "Cat" },
    },
}
    .attachTo(this);
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Slider { { 1, 10 }, 3 }.bind([this](wxCommandEvent& event) {
        SetStatusText(std::to_string(event.GetInt()));
    }),
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

Increased Safety

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        Widget<wxTextCtrl> { "Dog",
            wxSizerFlags(1).Expand().Border() },
        Widget<wxButton> { "Right" },
    },
    HSizer {
        Widget<wxButton> { "Left" },
        Widget<wxStaticText> { "Cat" },
    },
}
    .attachTo(this);
```

```
using namespace DeclarativeUI;
// Create and layout the controls.
VSizer {
    wxSizerFlags().Expand().Border(),
    HSizer {
        TextCtrl { wxSizerFlags(1).Expand().Border() }
            .withWidth(64),
        Button { "Right" }.bind([] {
            wxLogMessage("Hello world from wxWidgets!");
        }),
    },
    HSizer {
        Button { "Left" },
        Text { "Cat" },
    },
    Slider { { 1, 10 }, 3 }.bind([this](wxCommandEvent& event) {
        SetStatusText(std::to_string(event.GetInt()));
    }),
    Button { wxID_EXIT, "Exit" },
}
    .attachTo(this);
```

Established consistency

```
template <typename W>
struct Widget {
private:
    wxWindowID id;
    wxPoint position = wxDefaultPosition;
    wxSize size = wxDefaultSize;
    std::optional<wxSizerFlags> flags;
    std::optional<Handler> boundedHandler;
};
```

```
struct Text : details::Widget<Text> {
private:
    std::string str;
};
```

```
struct TextCtrl : details::Widget<TextCtrl> {
private:
    std::string str;
};
```

```
struct Button : details::Widget<Button> {
private:
    std::string str;
};
```

```
struct Slider : details::Widget<Slider> {
private:
    std::pair<int, int> range;
    int value;
};
```

Shifting to be a Library Writer

- Understand the problem domain.
- Make your designs consistent, safe, and easy to use.
- Anticipate your users needs.

Thank You

Questions?